

```

/*
 * P_Culiar.c - Reductor Teosófico con Captura de Teclas
 * Compilar: gcc P_Culiar.c -o P_Culiar -Wall
 * Ejecutar: ./P_Culiar
 *
 * Uso: Escribir un número y presionar SHIFT+F7
 * Ejemplo: 169[SHIFT+F7] → 169 [1+6+9] > R16 [1+6] > S7
 */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <termios.h>
#include <unistd.h>
#include <ctype.h>
#include <time.h>

// === TABLA DE ARQUETIPOS ===
const char* ARQUETIPO[] = {
    [1] = "Unidad/Inicio",
    [2] = "Dualidad/Equilibrio",
    [3] = "Trinidad/Expresión",
    [4] = "Orden/Estabilidad",
    [5] = "Voluntad/Poder (G7)",
    [6] = "Armonía/Unión",
    [7] = "Sabiduría/Intuición",
    [8] = "Infinito/Ciclo",
    [9] = "Compleitud/Cierre"
};

const char* MAESTRO[] = {
    [11] = "Maestro Intuición",
    [22] = "Maestro Constructor",
    [33] = "Maestro Maestro"
};

// === CONFIGURACIÓN DE TERMINAL ===
struct termios original_termios;

void modo_raw() {
    struct termios raw;
    tcgetattr(STDIN_FILENO, &original_termios);
    raw = original_termios;
    raw.c_lflag &= ~(ECHO | ICANON);
    raw.c_cc[VMIN] = 1;
    raw.c_cc[VTIME] = 0;
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);
}

void restaurar_terminal() {
    tcsetattr(STDIN_FILENO, TCSAFLUSH, &original_termios);
}

// === REDUCCIÓN TEOSÓFICA ===
int reducir_a_semilla(int num, char *traza, int *semilla_final) {
    int pasos = 0;
    int actual = num;

    while (actual >= 10 && actual != 11 && actual != 22 && actual != 33) {
        int suma = 0;
        int temp = actual;
        char suma_str[256] = "[";

        // Calcular suma de dígitos

```

```

while (temp > 0) {
    int digito = temp % 10;
    suma += digito;
    temp /= 10;
}

// Construir traza visual
temp = actual;
int primero = 1;
// Reconstruir dígitos en orden correcto
char digitos[64];
int idx = 0;
while (temp > 0) {
    digitos[idx++] = (temp % 10) + '0';
    temp /= 10;
}
for (int i = idx - 1; i >= 0; i--) {
    if (!primero) strcat(suma_str, "+");
    char d[2] = {digitos[i], '\0'};
    strcat(suma_str, d);
    primero = 0;
}
strcat(suma_str, "];");

// Determinar etiqueta R o S
char etiqueta[32];
if (suma < 10 || suma == 11 || suma == 22 || suma == 33) {
    sprintf(etiqueta, " > S%d", suma);
} else {
    sprintf(etiqueta, " > R%d", suma);
}

// Agregar a la traza
char paso_str[512];
sprintf(paso_str, " %s%s", suma_str, etiqueta);
strcat(traza, paso_str);

actual = suma;
pasos++;
}

*semilla_final = actual;
return pasos;
}

// === OBTENER ARQUETIPO ===
const char* obtener_arquetipo(int semilla) {
    if (semilla >= 1 && semilla <= 9) {
        return ARQUETIPO[semilla];
    } else if (semilla == 11 || semilla == 22 || semilla == 33) {
        return MAESTRO[semilla];
    }
    return "Desconocido";
}

// === GUARDAR EN LOG ===
void guardar_log(int numero_original, int semilla, const char *traza_completa) {
    FILE *log = fopen("p_culiar.log", "a");
    if (!log) return;

    time_t ahora = time(NULL);
    char timestamp[64];
    strftime(timestamp, sizeof(timestamp), "%Y-%m-%d %H:%M:%S",
    localtime(&ahora));

```

```

    fprintf(log, "[%s] Entrada: %d | Semilla: %d | Arquetipo: %s\n",
            timestamp, numero_original, semilla, obtener_arquetipo(semilla));
    fprintf(log, "  Traza: %d%s\n\n", numero_original, traza_completa);

    fclose(log);
}

// === DETECTAR SHIFT+F7 ===
int detectar_shift_f7() {
    // SHIFT+F7 en VT100 es: ESC [ 1 8 ; 2 ~
    char seq[16] = {0};
    int i = 0;

    // Leer secuencia de escape
    while (i < 15) {
        char c = getchar();
        seq[i++] = c;
        if (c == '~' || c == 'Q') break;
    }

    // Verificar si es SHIFT+F7
    return (strstr(seq, "18;2~") != NULL || strstr(seq, "18;") != NULL);
}

// === PROGRAMA PRINCIPAL ===
int main() {
    char buffer[1024] = {0};
    int pos = 0;

    printf("\n");
    printf("  P_CICULAR - Reductor Teosófico Neurobitrónico\n");
    printf("  Escribe números y presiona SHIFT+F7 para expandir\n");
    printf("  Ctrl+C para salir\n");
    printf("\n\n");

    modo_raw();

    while (1) {
        char c = getchar();

        // Ctrl+C
        if (c == 3) break;

        // ESC (posible tecla especial)
        if (c == 27) {
            if (detectar_shift_f7()) {
                // Buscar número hacia atrás
                int i = pos - 1;
                char num_str[64] = {0};
                int num_len = 0;

                while (i >= 0 && isdigit(buffer[i])) {
                    num_str[num_len++] = buffer[i];
                    i--;
                }

                if (num_len > 0) {
                    // Invertir el número
                    char num_final[64] = {0};
                    for (int j = 0; j < num_len; j++) {
                        num_final[j] = num_str[num_len - 1 - j];
                    }
                }
            }
        }
    }
}

```

```

        long numero = atol(num_final);
        char traza[1024] = {0};
        int semilla = 0;

        reducir_a_semilla(numero, traza, &semilla);

        // Borrar el número original
        for (int j = 0; j < num_len; j++) {
            printf("\b \b");
        }

        // Imprimir resultado
        printf("%ld%s", numero, traza);
        printf(" [%s]", obtener_arquetipo(semilla));
        fflush(stdout);

        // Guardar en log
        guardar_log(numero, semilla, traza);

        // Actualizar buffer
        pos -= num_len;
        char resultado[1024];
        snprintf(resultado, sizeof(resultado), "%ld%s [%s]", numero,
traza, obtener_arquetipo(semilla));
        for (int j = 0; resultado[j]; j++) {
            buffer[pos++] = resultado[j];
        }
    }
    continue;
}

// Backspace
if (c == 127 || c == 8) {
    if (pos > 0) {
        pos--;
        printf("\b \b");
        fflush(stdout);
    }
    continue;
}

// Caracteres imprimibles
if (c >= 32 && c <= 126) {
    buffer[pos++] = c;
    putchar(c);
    fflush(stdout);
}

restaurar_terminal();
printf("\n\n[LOG] Sesión guardada en p_culiar.log\n");
return 0;
}

```